
Topic 01: The Illusion of Certainty: Bayes' Theorem

Statistical Foundations & Probability

TOPIC 01

The Illusion of Certainty: Bayes' Theorem

Rare events, base rate fallacy, conditional probability geometry, and expectation.

Cognitive Objectives

By the end of this topic, learners will be able to:

- **Deconstruct the Base Rate Fallacy** in clinical diagnostics, fraud detection, and rare-event anomaly modeling.
- **Partition probability sample spaces** geometrically using joint, marginal, and conditional frequency trees.
- **Track belief updates visually** via the $\text{Prior} \times \text{Likelihood} \rightarrow \text{Posterior}$ Bayesian engine.
- **Implement a Bayesian belief updater** in vectorized NumPy for real-time sequential telemetry.

The Rare Event Paradox

The Production Dilemma: Fraud Detection

- Base rate of fraudulent transactions: 1% ($P(F) = 0.01$).
- ML Classifier Accuracy: 95% Sensitivity, 95% Specificity (5% False Positive Rate).
- An alert fires on Transaction #84920!
- **Socratic Probe:** What is the probability that this transaction is **actually** fraudulent?
- **Intuitive Guess:** “95%!” → **Completely wrong!**

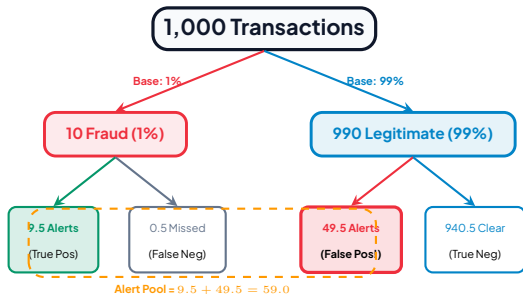
The “Aha!” Discovery: Base Rate Dominance

Consider 1,000 transactions:

- **10 Fraudulent:** 9.5 trigger True Alert, 0.5 Missed.
- **990 Legitimate:** 5% of 990 = **49.5** trigger **False Alerts!**
- Total Alerts Fired: $9.5 + 49.5 = 59.0$.
- True Probability: $\frac{9.5}{59.0} \approx 16.1\%$

PUNCHLINE Base rates dominate high-accuracy test outcomes.

Hero Visualization: Frequency Tree Partitioning



Bayes' Theorem Formalized

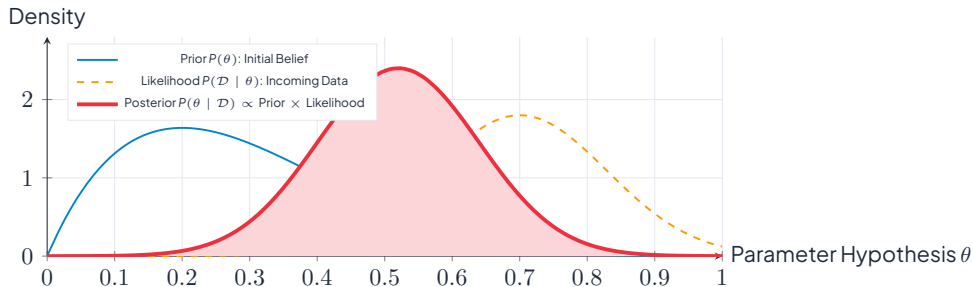
- Bayes' Formula:

$$P(F | +) = \frac{P(+ | F)P(F)}{P(+ | F)P(F) + P(+ | \neg F)P(\neg F)}$$

- Numerator (True Signal):
 $0.95 \times 0.01 = 0.0095$.
- Denominator (Total Evidence):
 $0.0095 + (0.05 \times 0.99) = 0.0590$.
- Posterior Probability:

$$P(F | +) = \frac{0.0095}{0.0590} = 16.1\%$$

Hero Visualization: Prior $\times$ Likelihood $\rightarrow$ Posterior



PUNCHLINE

Today's posterior becomes tomorrow's prior in recursive learning systems.

Production Checkpoint: Vectorized Bayesian Filter

Sequential Bayesian Inference

For hypothesis space $\Theta = [\theta_1, \dots, \theta_K]$:

- **Prior:** $p_0 \in \mathbb{R}^K$ (Uniform or historical).
- **Update Rule:**

$$p_t = \frac{p_{t-1} \odot L(\text{obs} \mid \Theta)}{\sum_{k=1}^K p_{t-1,k} L(\text{obs} \mid \theta_k)}$$

- Exact, closed-form updates without MCMC sampling for discrete parameter spaces.

Vectorized NumPy Implementation

```
1 import numpy as np
2
3 class DiscreteBayesianFilter:
4     def __init__(self, hypotheses: np.ndarray):
5         self.hypo = hypotheses
6         # Start with uniform prior
7         self.prior = np.ones_like(hypotheses) / len(
            hypotheses)
8
9     def update(self, likelihoods: np.ndarray) -> np.
        ndarray:
10         """likelihoods: P(Data | hypo_k)"""
11         unnormalized = self.prior * likelihoods
12         evidence = np.sum(unnormalized)
13         self.prior = unnormalized / evidence
14         return self.prior
```